

CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool

Marko Ivanković
Universität Passau
Passau, Germany
marko@ivankovic.me

ABSTRACT

A source-code diffing tool needs an accurate, efficient way to map one abstract syntax tree (AST) onto another. Decades of research have given the field exact tree-edit-distance algorithms, Zhang-Shasha and APTED, and source-code-specific heuristics, GumTree, plus a newer generation of practical, tree-sitter-based tools, difftastic and diffsitter. This paper presents CODEDIFF, a syntax-aware code-diffing tool that builds directly on this line of work, combining APTED’s exact tree edit distance with GumTree’s bottom-up and move-recovery heuristics and Myers’ $O(ND)$ sequence diff into one seven-phase matching pipeline. CODEDIFF’s own contribution is not a new matching algorithm but two engineering disciplines applied on top of this foundation. First, its performance and robustness targets come from an empirical study of 100 real-world open-source repositories, about 1.35 million files in 30 languages, not from worst-case asymptotic analysis alone. Second, each optional heuristic pass ships only if an ablation study shows a measurable accuracy gain on a corpus of 98 real-world diffs with human-verified ground truth, so that a technique’s published benefit is confirmed in CODEDIFF’s own pipeline rather than assumed to transfer unchanged. On that same ground-truth corpus, CODEDIFF maps 99.8% of AST nodes correctly, and reaches that accuracy at a speed and reliability suited to everyday, interactive use—read alongside GumTree, difftastic, and diffsitter not as a replacement for any of them, but as one way of combining what each already does well.

CCS CONCEPTS

• **Software and its engineering** → **Software maintenance tools**; *Empirical software validation*.

KEYWORDS

source code differencing, syntax-aware diffing, empirical software engineering, source code tree edit distance

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference ’26, June 03–05, 2026, Ludbreg, Croatia

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/26/06

<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Marko Ivanković. 2026. CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference’26)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Code diffing, the process of comparing two pieces of source code and computing the set of operations that transforms one into the other, underlies modern version control and, especially, modern code review. During a review, developers read the diff to see what changed and what stayed the same, so the quality of the diff directly shapes the quality of the review.

The standard diff used by almost all developers and platforms today is line-based: the file is treated as a sequence of text lines, and the tool computes a shortest edit script between the two sequences using three operations, insert, delete, and update. This approach is computationally cheap and fast even on modest hardware. However, most implementations offer no move operation at all, and the tools that do support one support it only in limited ways. A whole class of common changes—extracting code into a reusable function, wrapping a block in a condition, reordering declarations—is ultimately a relocation of existing code, and without a move operation every such change must be rendered as a full deletion plus a full insertion. Reconstructing the correspondence between the deleted and inserted halves is then left to the reviewer, by eye, which is exactly the kind of mechanical, attention-taxing work humans do least reliably.

One way to compute diffs with a genuine move operation is to compare the code’s abstract syntax tree (AST) instead of its raw text. An AST diff maps nodes of one tree onto nodes of the other; a move is a mapping whose nodes match but whose ancestry changed. Tree diffing is a well-studied field, and Section 2 surveys the work this paper builds on. In practice, however, tree diffing has seen little adoption, and we argue the central reason is cost: the best exact tree-diffing algorithms are $O(n^3)$ in tree size, and Section 3 measures directly how often a whole-tree computation at real-world file sizes exceeds an interactive time budget.

This paper presents CODEDIFF, a syntax-aware code-diffing tool that combines established algorithms with rigorous engineering practice to meet explicit performance and robustness targets. The targets come from an empirical study of real open-source projects—the sizes and shapes of their files and

of the changes developers actually make—rather than from worst-case analysis alone.

This paper makes the following contributions:

- It reports an empirical study of 100 real-world open-source repositories, showing that the worst-case inputs tree-diffing algorithms are analyzed against are rare in practice, but that the tail is real and large enough that a production tool must handle inputs beyond what a whole-tree tree-edit-distance computation can process within an interactive budget.
- It reports a quantified comparison of four established diffing tools—GumTree, Unix `diff`, `difftastic`, and `diffsitter`—against human-authored ground-truth mappings of 98 real-world code changes.
- It reports a leave-one-out ablation study identifying which shapes of change require a dedicated matching heuristic, and showing that a heuristic’s published benefit does not automatically transfer into a different pipeline evaluated against a different corpus.
- It details CODEDIFF, a seven-phase diffing pipeline that combines APTED’s exact tree edit distance, GumTree’s source-code-specific heuristics, and Myers’ sequence diff into one production-engineered tool meeting the targets the empirical study sets.

2 BACKGROUND

This section introduces the algorithms and tools the rest of the paper depends on: the algorithms CODEDIFF builds its pipeline from, and the tools it is evaluated alongside. Section 6 returns to the closest prior work in more depth once the paper’s own results are in hand.

2.1 Algorithms

Zhang-Shasha [9] gives the classic dynamic-programming algorithm for ordered tree edit distance. Given two rooted, ordered trees, it computes the minimum-cost sequence of node insertions, deletions, and relabelings that transforms one tree into the other, together with the mapping that realizes that cost. It is exact, but its keyroot decomposition does repeated work across overlapping subproblems. Later algorithms reduce this overhead while keeping the same exact result.

APTED [8] is the state-of-the-art exact tree-edit-distance algorithm. It improves on Zhang-Shasha by choosing, for each subproblem, the cheapest of three decomposition strategies (left path, right path, or heaviest inner path), rather than a single fixed strategy. This choice provably minimizes the total work across the whole computation. APTED still costs $O(n^3)$ in the worst case for general trees, the same asymptotic bound as Zhang-Shasha, but its real-world runtime is markedly lower, because most real trees are far from the adversarial shape that bound assumes.

GumTree [4] targets source code directly, not general trees. Instead of one global tree-edit-distance computation, it runs a top-down phase that greedily matches large isomorphic subtrees, then a bottom-up phase that matches

remaining container nodes by the Dice coefficient of their already-matched descendants. GumTree also introduces a distinct edit operation, move, and a recovery pass that pairs up deleted and inserted identical subtrees the top-down and bottom-up phases missed. This design favors speed over exactness, at the scale real source files require.

Myers’ $O(ND)$ algorithm [7] solves a different, more restricted problem: the shortest edit script between two flat sequences, not two trees. It underlies most line-based diff tools, including Unix `diff`. Its cost scales with the size of the edit script, not the size of the inputs, so it stays fast when two sequences are similar, exactly the common case for a version-controlled file.

2.2 Tools

difftastic [5] and **diffsitter** [3] are a newer generation of practical, tree-sitter-based diffing tools, the same parsing layer CODEDIFF itself is built on. Both prioritize a diff a human reads comfortably over an exact node-to-node mapping: `difftastic` reduces the parse tree to an s-expression and aligns it with its own structural algorithm, and `diffsitter` applies a related tree-sitter-based alignment over a smaller, hand-picked set of compiled-in grammars. Neither computes a full tree-edit-distance mapping the way APTED or GumTree do, so neither directly targets the ground-truth-accuracy metric this paper measures against in Section 5—a difference in design goal, not a shortcoming, and the reason we include both as companions in the same problem space rather than as head-to-head competitors on identical terms.

3 PROBLEM SPACE

Formal treatments of the algorithms in Section 2 report worst-case asymptotic complexity: APTED and Zhang-Shasha are both $O(n^3)$ for general trees, where n is tree size. This analysis is necessary, but it is not sufficient for a production tool. GumTree’s own documentation states that it targets research use, not production deployment at scale—a reasonable scope for its own goals, but one that leaves open a question every production diff tool must answer:

RQ1 (Scale). What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation—the algorithmic core every exact, syntax-aware diff tool is built on—complete within a one-second budget?

A tool engineered only against a worst-case bound risks two failure modes. It can over-engineer for inputs that never occur in practice. It can under-engineer for a real tail its asymptotic analysis never quantified. Answering RQ1 needs measurement, not further asymptotic analysis.

We measure this directly. We built a corpus of 100 open-source repositories, drawn from the Gentoo Linux distribution’s package list and popular repositories on GitHub. We cloned every repository in full and parsed every file with Tree-sitter, recording byte size, line count, language, and AST node count per file. The full methodology, and every

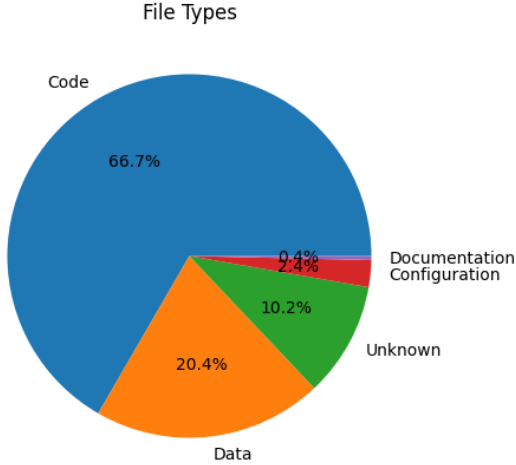


Figure 1: Distribution of file types across the corpus.

Table 1: Per-file size percentiles across all languages, code files only.

Metric	p50	p90	p99	p99.9	Max
Bytes	1,508	13,945	90,132	464,325	23,949,786
LOC	42	378	2,244	7,818	65,563
AST nodes	224	2,868	18,553	70,828	573,334

per-language number, is reproducible from the project’s research repository; here, we summarize the numbers that shape CODEDIFF’s design.

The corpus has 1,347,490 files across 30 languages. About two-thirds of all files are code. The rest is configuration, data, and documentation (Figure 1). Restricted to code files, Table 1 reports size percentiles in bytes, lines of code, and AST nodes.

The largest file in the corpus has 573,334 AST nodes. Byte size and AST node count have a strong correlation, Pearson $r = 0.8794$, which simplifies to $|\text{AST}| \approx \text{bytes}/5$ for quick estimates. Arafat and Riehle report that 47.5% of all commits touch 10 lines or fewer [1], while commits of over 10,000 lines still occur.

Measurement. To answer RQ1’s percentage directly, we sampled 7,340 real (before, after) file pairs from single-parent commits in these repositories, stratified per language and per size bucket so that large files are represented rather than drowned out by the far more common small ones. We ran a single whole-tree tree-edit-distance computation on every pair—CODEDIFF’s own APTED implementation, applied to the full before and after trees with none of CODEDIFF’s pipeline heuristics—under a hard one-second budget enforced by terminating the computation’s process. This implementation includes an optimization that makes it faster than a textbook APTED, so the failure rates below are a lower

bound on how often a whole-tree computation misses the budget, not an upper bound.

Figure 2 shows the fraction of pairs completed within one second, per size bucket, split by what kind of artifact the file is: code (5,765 pairs across general-purpose programming languages), and configuration, data, and markup formats (1,575 pairs). The split matters because the two populations behave differently—blended, the headline number would move with how many configuration files a sample happens to contain, independently of anything the algorithm does to source code.

RA1. For source code, a whole-tree computation is dependable only for small files: it completes within one second for 99.9% of code pairs of 10–30 lines, but for only 9.5% of code pairs of 100–300 lines—well inside the range of ordinary source files (Table 1)—and never recovers above one third in any larger bucket. Across all sampled code pairs, 40.6% complete in time; this sample-weighted aggregate deliberately over-represents large files and therefore understates the rate a uniform sample of real commits would show, but the per-bucket rates carry the answer regardless of weighting. The common case is genuinely cheap—most files and most commits are small—but the tail is real: files up to 573,334 AST nodes occur in this corpus, and Arafat and Riehle’s data shows commits of over 10,000 lines occur in practice. A production diff tool must be engineered for both regimes, and cannot rely on a whole-tree computation alone for either. Section 4 turns this answer into CODEDIFF’s concrete design targets.

4 CODEDIFF

The measurements in Section 3 give CODEDIFF two concrete, named design targets, stated in place of an asymptotic bound.

Robust: CODEDIFF must handle every file up to the measured maximum, 573,334 AST nodes, with headroom above that number. **Fast:** CODEDIFF must compute a diff in under 400 ms for 99.99% of real-world commits, the range that covers the overwhelming majority of commits per Arafat and Riehle’s measurement. Section 5 reports how well CODEDIFF meets both targets in practice.

CODEDIFF matches two ASTs in seven phases. Each phase either resolves part of the mapping directly or narrows the residual left for the next phase. Later phases only ever see nodes earlier phases left unmatched.

Phase 1, hash-based descent. CODEDIFF matches subtrees by three hash variants, in order: byte-identical subtrees, then same-shape subtrees with any leaf value, then import statements matched by normalized path. Each variant claims what it can before the next runs. This phase is CODEDIFF’s own design, motivated directly by Section 3’s commit-size data: since most commits touch a small fraction of a file’s lines, most of a typical file’s AST is byte-identical between before and after. Resolving that identical majority with cheap hashing, before any tree-edit-distance computation runs at all, avoids paying that computation’s cost on content that never changed.

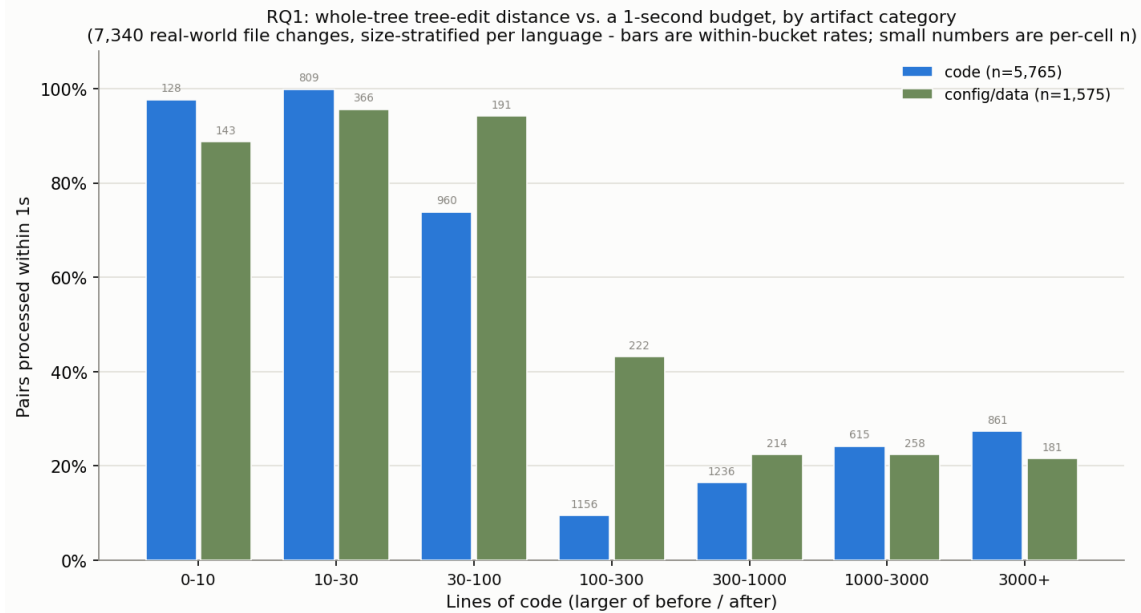


Figure 2: Fraction of sampled file pairs a whole-tree tree-edit-distance computation completes within a one-second budget, per size bucket and artifact category.

Phase 2, contextual exact matching. Comments that immediately precede an already-matched node, and diagnostic statements (logging calls, assertions, panics) that are byte-identical on both sides, get matched directly. Both are cheap, high-confidence matches that reduce the work left for the tree-edit-distance phases below.

Phases 3 and 5, bottom-up expansion. This is GumTree’s bottom-up matching heuristic: once most of an unmatched node’s descendants already match the descendants of some unmatched node on the other side, by a Dice-coefficient threshold, match the two nodes to each other too, even though nothing matched their own labels directly. CODEDIFF runs this pass twice, once before phase 4 and once after, since phase 4 produces more matched descendants for the second pass to vote on. Section 5 reports that, once combined with the rest of this specific pipeline, this heuristic does not carry over the accuracy gain GumTree’s own evaluation found for it, so CODEDIFF ships it disabled by default—a property of this pipeline’s measured results, not of the heuristic’s origin.

Phase 4, syntax-aware subtree matching. This phase generalizes three matching strategies into one engine: matching by fully-resolved name (handling overloads and duplicate names across scopes), a greedy cost-estimate matcher for containers with no name or arm structure, such as an `if` block or a loop body, and a Jaccard-similarity matcher for flow-control constructs whose case arms mostly agree. Before any of these run, large flat subtrees, such as a macro’s token list or a big flat argument list, are pre-matched directly with Myers’ algorithm [7], since a flat sequence has no tree structure worth exploiting—exactly the structures that drive Section 3’s measured AST-size tail. Each accepted pair is

then handed to APTED (below) to compute its exact internal edit script.

Phase 6, final APTED. Whatever remains unmatched after phases 1 through 5 is handed to APTED [8] for one whole-residual tree-edit-distance computation, giving an exact, cost-minimal mapping over that remainder. When the residual is too large for this to finish inside CODEDIFF’s speed target, a cheaper Myers-LCS alignment [7] over the residual substitutes for full APTED instead, trading a small amount of match quality for a bounded worst-case latency. This fallback exists directly because of the measured size tail in Section 3: without it, the rare very large residual blows the 400 ms target outright.

Phase 7, move-detection recovery. The final pass. Ordered tree edit distance cannot express a subtree that relocated across a matched boundary as anything but a delete plus an insert. This phase, adapted directly from GumTree’s recovery-mappings phase [4], pairs up whole subtrees left fully deleted on one side and fully inserted on the other, when they are byte-identical and large enough to rule out coincidence. Unlike phase 3/5’s bottom-up heuristic, Section 5 finds this heuristic measurably helps, so it ships enabled by default.

Zhang-Shasha [9] plays no role in this pipeline as a shipped algorithm. CODEDIFF’s own test suite instead implements it from scratch as an independent oracle, and fuzz-tests thousands of random trees to confirm that APTED’s result always matches Zhang-Shasha’s, bit for bit. This checks the pipeline’s most correctness-critical component against a second, independently-implemented ground truth, not only against itself.

5 EVALUATION

Dataset. CODEDIFF’s accuracy claims rest on a corpus of 98 fixtures. Each fixture pairs a before and after version of a source file, representing a realistic code change—a bug fix, a refactor, an added feature, or a performance-motivated rewrite—across many languages. Some fixtures are captured directly from a specific commit in a real open-source repository. Others are curated examples of a common, realistic change pattern. For every fixture, a human annotator used a purpose-built tool, `human_solver`, to walk the before and after syntax trees side by side and label every node with one of seven operations: identical, update, match-but-not-identical, insert, insert-with-children, delete, and delete-with-children. Nodes are addressed by a path of kind and sibling position, not by parser-internal node identity, since that identity is not stable across separate parses of the same file. This human-authored mapping is the ground truth every number in this section is measured against.

This section answers two further research questions, building on RQ1’s finding that a real, non-negligible tail of large files and commits exists alongside a small common case, and then evaluates CODEDIFF itself—the paper’s system contribution—against the design targets stated in Section 4.

RQ2 (Tool accuracy). How accurately do current state-of-the-art code-diffing tools recover real-world code changes, measured against human-authored ground-truth mappings of those changes?

RQ3 (Shapes and heuristics). Which shapes of real-world change—renames, moves, restructurings—require a dedicated matching heuristic, and do heuristics shown effective in one pipeline and corpus retain that value when reused in a different pipeline and evaluated against a different corpus?

Comparison tools. We place CODEDIFF alongside four established tools, on the same 98-fixture corpus: GumTree (v4.0.0-beta8) [4], Unix `diff`, and the two tree-sitter-based tools from Section 2, `difftastic` [5] and `diffsitter` [3].

This comparison measures one specific property: agreement with the human-authored ground-truth mapping above. `difftastic` and `diffsitter` deliberately optimize for something this metric does not capture, a diff that reads comfortably to a human, not a mapping that reproduces a ground truth line for line, so a lower score here reflects a different design goal, not a defect. GumTree remains the foundation two of CODEDIFF’s own pipeline phases build on directly (Section 4), and its broader language and backend flexibility stays valuable well beyond what this single corpus measures. Read what follows as a report of where combining these approaches into one production-engineered, ablation-validated pipeline lands, not as a ranking of which tool is best.

AST-node accuracy. Measured directly against the human mapping, CODEDIFF maps 389,398 of 390,142 AST nodes correctly across the 98 fixtures, 744 mismatches, or 99.8% node-level accuracy.

Line-level agreement. None of the other four tools report an AST-node mapping, so we compare all five on a common

Table 2: Line-level mismatches against human ground truth, each tool scored on its own applicable subset of the 98-fixture corpus.

Tool	Fixtures	Mismatched lines	Rate
CODEDIFF	98	479	0.887%
Unix <code>diff</code>	98	570	1.055%
GumTree	97	637	1.198%
<code>difftastic</code>	98	877	1.624%
<code>diffsitter</code>	83	960	1.897%

ground: which source lines each tool marks as touched, against the same human mapping (Table 2, Figure 3). Coverage varies: `diffsitter`’s compiled-in grammar set matches 83 of the 98 fixtures’ languages, GumTree’s generator set matches 97, and the rest match all 98—each tool’s own row is scored only on its own applicable subset, not padded with unscored fixtures.

RA2. Among the four pre-existing tools (Table 2, excluding CODEDIFF’s own row), line-level mismatch rates against the human mapping range from 1.055% (Unix `diff`) to 1.897% (`diffsitter`). Two results stand out. First, every tool already agrees with the human mapping on the overwhelming majority of lines: even the highest mismatch rate leaves more than 98% of lines in agreement, so the question that separates tools is how they handle the small, structurally interesting remainder. Second, no AST-aware tool among the four beats plain line-based `diff` on this measure. Syntax-awareness alone does not guarantee that a tool recovers a change the way a human reads it—the gap is in which heuristics act on the syntax tree, which RQ3 turns to next.

Ablation study. RQ3 asks which shapes of change earn a dedicated heuristic. CODEDIFF has four optional heuristic passes, each targeting a specific shape and each behind its own flag: import-node normalization (renamed or re-ordered import paths), flow-control arm matching (restructured `if/switch` arms), bottom-up expansion (containers whose children already match, phases 3 and 5), and move-detection recovery (whole subtrees relocated across a matched boundary, phase 7). A leave-one-out ablation study measured each pass’s real contribution on the 98-fixture corpus above, by disabling one pass at a time and comparing the resulting mismatch count against a baseline with all four enabled.

Table 3 reports the result. Import-node normalization and flow-control arm matching, two of CODEDIFF’s own heuristics, both turned out net-negative for accuracy on this corpus once combined with the rest of the pipeline. Bottom-up expansion, adapted from GumTree, shows the same pattern here—not because the technique is unsound (GumTree’s own evaluation, on GumTree’s own pipeline and corpus, found it valuable), but because a heuristic’s net effect depends on what the rest of the pipeline already resolves before that heuristic gets its turn, and CODEDIFF’s earlier phases leave GumTree’s bottom-up phase less new ground to help with. Move-detection recovery, GumTree’s other contribution to this pipeline, is the one of

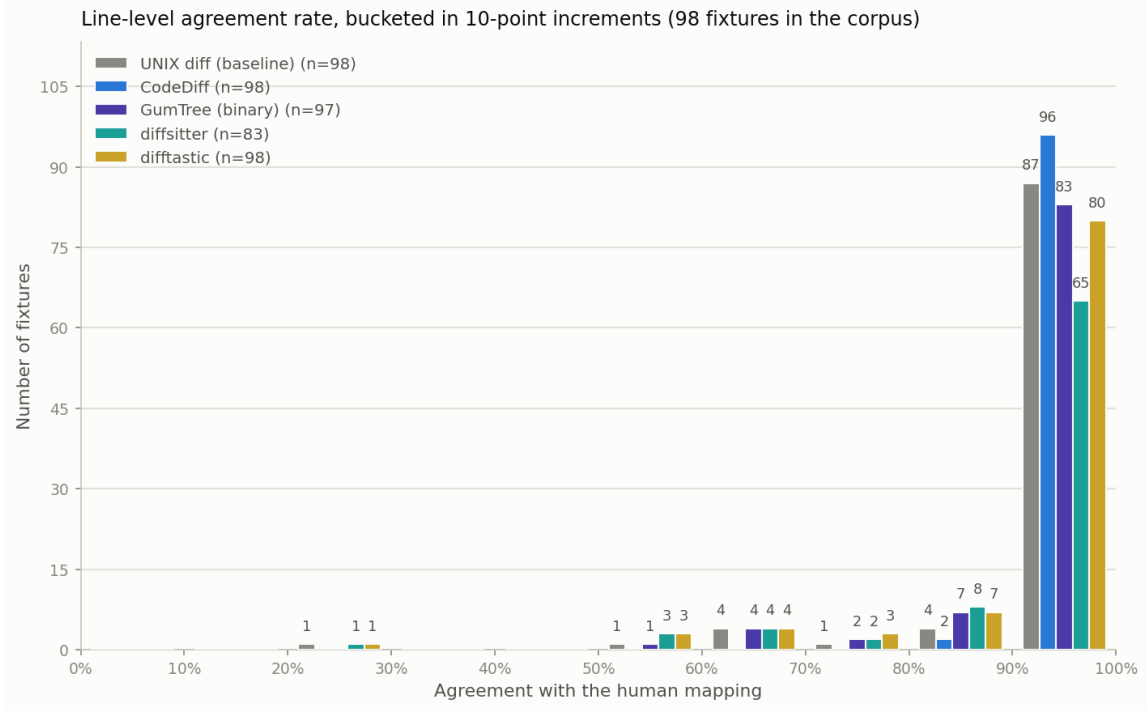


Figure 3: Line-level agreement rate with the human mapping, bucketed in 10-point increments, all five tools on the same axis. Each series’ own n reflects its applicable subset of the corpus.

Table 3: Ablation study: accuracy change from disabling each optional pass, against a baseline with all four enabled. A negative number means disabling the pass improved accuracy, fewer mismatches. A positive number means disabling it hurt accuracy. The two passes GumTree contributes are marked (G).

Pass	Δ if disabled	Ships
Import-node normalization	−89	disabled
Flow-control arm matching	−82	disabled
Bottom-up expansion (G)	−69	disabled
Move-detection recovery (G)	+3	enabled

the four that does carry over: it measurably helps here too, and ships enabled by default. The other three ship disabled.

RA3. On this corpus, the move is the one shape whose dedicated heuristic pays for itself: move-detection recovery is the only optional pass whose removal hurts accuracy, which matches the structural argument of Section 4—a relocated subtree is the one shape ordered tree edit distance cannot express except as a delete plus an insert, so no downstream computation can recover it without a dedicated pass. The heuristics targeting the other shapes are net-negative here, including bottom-up expansion, which GumTree’s own evaluation, on GumTree’s own pipeline and corpus, validated as helpful. A technique documented as effective in its original

paper is therefore not guaranteed to help inside a different pipeline evaluated against a different corpus. CODEDIFF measures this directly for every optional pass, rather than assuming a result from the literature transfers unchanged.

Production viability. The remaining measurements evaluate CODEDIFF itself—speed, then robustness—against the Fast and Robust targets stated in Section 4.

Speed. Table 4 and Figure 4 report wall-clock percentiles over the same fixtures, sorted fastest to slowest by median. We report GumTree twice: once invoked as a fresh process per fixture, the realistic cost of a single CLI invocation, and once run inside one persistent JVM across all fixtures, isolating JVM startup from GumTree’s own matching cost. difflsitter is the fastest AST-aware tool in this comparison at every percentile measured, a direct result of its narrower, hand-picked grammar set. difftastic sits close to CODEDIFF at the median and 90th percentile, but its tail is heavier: at the 99th percentile, difftastic is the slowest AST-aware tool here after GumTree invoked as a fresh process. Unix `diff` remains fastest overall at every percentile, since it never parses or compares tree structure.

These percentiles must be read against the right population. The Fast target of Section 4—400 ms for 99.99% of real-world commits—is a claim about the real distribution of commits, in which 47.5% touch 10 lines or fewer [1]. The fixture corpus is not that distribution: it deliberately concentrates hard, structurally interesting changes, because easy changes exercise nothing worth measuring. CODEDIFF’s 99th

Table 4: Wall-clock time percentiles, milliseconds, sorted fastest to slowest by median.

Tool	p50	p90	p99
Unix <code>diff</code>	2.6	3.1	15.8
diffsitter	4.5	35.3	122.5
CODEDIFF	9.6	268.3	1364.2
GumTree (warm JVM)	42.6	440.4	2418.5
diffastic	55.3	109.6	2704.5
GumTree (cold, per-invocation)	442.1	978.1	2832.8

Table 5: Timing noise per series: per-fixture coefficient of variation across repeated measurements of the same fixture (lower means a single run is more trustworthy on its own).

Series	<i>n</i>	Median CoV	p90 CoV
TreeSitter parse (lower bound)	98	9.7%	20.1%
UNIX diff (baseline)	98	6.2%	9.5%
CodeDiff	98	2.1%	12.3%
GumTree (binary)	97	3.2%	5.9%
GumTree (warm JVM)	97	3.1%	9.6%
diffsitter	83	4.4%	8.0%
diffastic	98	4.0%	27.3%

percentile on this corpus (1364.2ms) is therefore a stress figure on an adversarially weighted sample, not the target metric; its median (9.6ms) and 90th percentile (268.3ms) on the same hard sample sit well inside the budget. Validating the 99.99% claim on a uniformly drawn commit sample remains future work, and we state it here as a design target rather than a measured result.

Table 5 reports a companion measurement: how much any single timing run can be trusted, the per-fixture coefficient of variation across repeated measurements of the same fixture. This is what motivated repeating every timing number in this paper across multiple runs rather than trusting a single measurement.

Robustness. We sampled 1,000 real (repository, commit, file) pairs from real Rust repositories and ran every pair through CODEDIFF, up to a 16,000 combined AST-node cap and a 120-second timeout per pair. Of the 1,000 sampled pairs, 386 were unavailable locally because the repository’s history had since been rewritten, not a CODEDIFF failure, and 107 exceeded the node cap and were skipped before CODEDIFF ever ran. CODEDIFF completed all 507 remaining pairs with zero panics and zero timeouts. This is a sampled, bounded-scale result, not a claim over the full 100-repository corpus, but it is consistent with CODEDIFF’s stated Robust target in Section 4.

Summary. Combining established techniques rather than inventing new ones, CODEDIFF reaches 99.8% AST-node accuracy against human-verified ground truth, the lowest line-level mismatch rate of the five tools compared, wall-clock

latency faster than GumTree and diffastic at the median, and zero panics or timeouts across the sampled robustness evaluation. Building a production-grade syntax-aware diffing tool did not require a new matching algorithm, only careful combination and measurement of existing ones.

6 RELATED WORK

Tree-diffing tools. GumTree [4] remains the reference point for source-code tree diffing, and two of CODEDIFF’s pipeline phases adapt its heuristics directly (Section 4). The difference is one of goal: GumTree targets research use, with broad language and backend flexibility, while CODEDIFF narrows to a tree-sitter-only frontend and spends the saved generality on production targets—and, as the ablation in Section 5 shows, on validating each adopted heuristic inside its own pipeline rather than assuming the published benefit transfers. diffastic [5] and diffsitter [3] share CODEDIFF’s parsing layer but optimize for a diff a human reads comfortably rather than for fidelity of the underlying node mapping; RA2 reflects that difference in design goal, not a defect in either tool. Unix `diff`, built on Myers’ algorithm [7], remains the baseline every one of these tools must justify itself against—on our corpus it is both the fastest tool measured and, at the line level, more accurate than every pre-existing AST-aware tool compared.

Empirical studies. Dyer et al. [2] and Lämmel et al. [6] mine AST nodes at scale to study API usage; Arafat and Riehle [1] measure the commit-size distribution of open-source software, the distribution CODEDIFF’s Fast target is stated against. This paper’s empirical study differs from these in purpose rather than method: it measures file- and AST-size distributions specifically to set, and then evaluate, the engineering targets of a diffing tool.

7 CONCLUSION AND FUTURE WORK

None of CODEDIFF’s matching algorithms are new. Its contribution is combining Zhang-Shasha’s and APTED’s exact tree edit distance, GumTree’s source-code-specific heuristics, and Myers’ sequence diff into a single pipeline, engineered and measured against the real distribution of source code rather than an assumed worst case, and validating every optional piece of that combination by measurement instead of by assumption. On the corpus in this paper, that combination reaches an accuracy, speed, and reliability comparable to or exceeding each individual predecessor it draws from, without displacing what any of them are independently good at: GumTree’s language and backend flexibility, diffastic’s and diffsitter’s speed and readability, Unix `diff`’s universality. The ablation result in Section 5 is the more general, transferable point: a heuristic’s value is a property of the pipeline and corpus it runs in, not only of the paper that introduced it. A production tool must measure that value directly, rather than assume it carries over.

Based on the lessons reported in this paper, we recommend the following to builders of syntax-aware developer tooling:

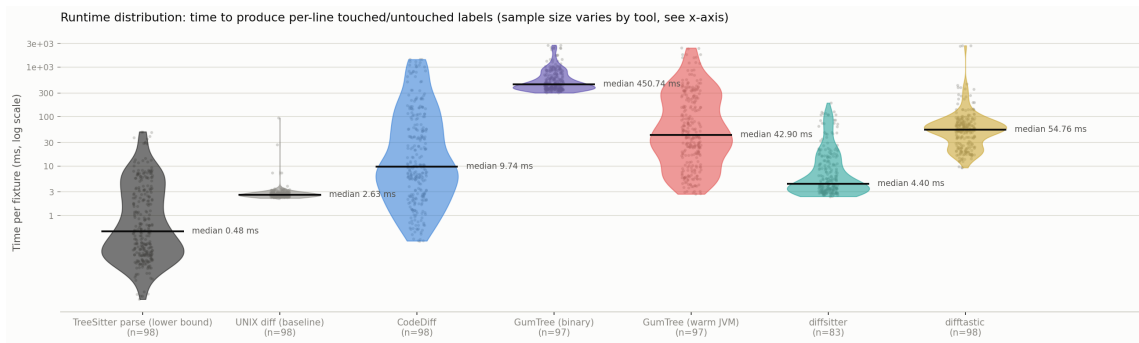


Figure 4: Full per-fixture runtime distribution (log scale), every individual repeated measurement plotted, alongside a tree-sitter-parse-only reference violin (the lower bound every AST-aware tool here must pay before its own work even starts).

- Measure every adopted heuristic inside your own pipeline, against your own corpus: of the two GumTree heuristics adopted here, one helped and one hurt, and only direct measurement told them apart.
- Engineer against the measured distribution of real inputs, not against the worst-case bound alone: the common case and the tail require different machinery, and both occur in practice.
- Keep an exact algorithm as an independent test oracle even when the shipped path is heuristic: CODEDIFF fuzz-checks its APTED implementation against an independently implemented Zhang-Shasha on thousands of random trees.

Future work includes widening the ground-truth and robustness corpora beyond Rust and the current 98 fixtures, extending the accuracy comparison to more languages GumTree also supports, re-running the ablation study as new heuristic passes are added to the pipeline, and quantifying the distribution of change shapes in the ground-truth corpus itself, so that RQ3's shape findings rest on measured frequencies rather than on the ablation alone.

REFERENCES

- [1] Osama Arafat and Dirk Riehle. 2009. The Commit Size Distribution of Open Source Software. In *2009 42nd Hawaii International Conference on System Sciences*. 1–8. <https://doi.org/10.1109/HICSS.2009.421>
- [2] Robert Dyer, Hridesh Rajan, Hoan Anh Nguyen, and Tien N. Nguyen. 2014. Mining billions of AST nodes to study actual and potential usage of Java language features. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 779–790. <https://doi.org/10.1145/2568225.2568295>
- [3] Afnan Enayet. 2020. difflsitter: A Tree-Sitter-Based AST Diff tool for Meaningful Diffs. <https://github.com/afnanenayet/difflsitter>. Accessed 2026-07-31.
- [4] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*. Västerås, Sweden, 313–324. <https://doi.org/10.1145/2642937.2642982>
- [5] Wilfred Hughes. 2018. Difftastic: A Structural Diff That Understands Syntax. <https://github.com/Wilfred/difftastic>. Accessed 2026-07-31.
- [6] Ralf Lämmel, Ekaterina Pek, and Jürgen Starek. 2011. Large-scale, AST-based API-usage analysis of open-source Java projects. In *Proceedings of the 2011 ACM Symposium on Applied Computing*. 1317–1324.
- [7] Eugene W. Myers. 1986. An O(ND) Difference Algorithm and Its Variations. *Algorithmica* 1, 2 (1986), 251–266. <https://doi.org/10.1007/BF01840446>
- [8] Mateusz Pawlik and Nikolaus Augsten. 2015. Efficient Computation of the Tree Edit Distance. *ACM Transactions on Database Systems* 40, 1, Article 3 (2015), 3:1–3:40 pages. <https://doi.org/10.1145/2699485>
- [9] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. <https://doi.org/10.1137/0218082>